RAPPORT D'AVANCEMENT / DÉMONSTRATION

Création de décors et d'éléments de décors pour concert de musique virtuel

Un premier atelier BabylonJS pour assembler, personnaliser et conserver une scène de concert.



Aya
Encadrant : Michel Buffa
Projet WAM JAM Party · Université Côte d'Azur

13 novembre 2026

ÉTAT DU LIVRABLE

Prototype de bureau fonctionnel et testé. Intégration WAM JAM Party et validation sur casque à poursuivre.

Capture réelle du prototype · Assemblage développé avec assistance IA

01 / SYNTHÈSE

Un atelier déjà manipulable

Le travail a franchi le stade de la spécification : une première expérimentation permet de composer et de retrouver un décor.

Le prototype est développé dans le laboratoire Hyppolite, sur la branche **aya-scene-et-decore**. Il réutilise la scène de concert existante et lui ajoute une page indépendante consacrée aux décors.

164
tests unitaires et intégration

23
nouveaux tests décors

16
objets dans l'export d'Aya

Ce qui est démontrable	Portée actuelle
Composer une scène	Configuration à un ou trois écrans ; catalogue de six types de décors.
Modifier un décor	Sélection, déplacement par gizmo, position, rotation, taille et couleur.
Régler les éclairages	Trois spots RGB initiaux ; puissance et représentation du faisceau.
Conserver son travail	Sauvegarde locale explicite et export/import du JSON du prototype.
Présenter un résultat	Rendu réel, scénarios de test, captures et parcours de démonstration.

La prochaine décision structurante

Confirmer avec l'encadrant le lieu d'intégration définitif et le contrat de chargement dans WAM JAM Party. Le format JSON actuel est propre à l'atelier : sa compatibilité avec l'application cible n'est pas encore démontrée.

Lecture du dossier : contexte et périmètre p. 3 ; rendus p. 4-6 ; architecture p. 7 ; validation p. 8 ; démonstration p. 9 ; suite du projet p. 10 ; références p. 11.

02 / BESOIN ET PÉRIMÈTRE

Concevoir un décor pour un atelier musical

La création de décors doit servir un environnement musical immersif et collaboratif existant.

WAM JAM Party représente des composants musicaux sous forme d'objets 3D interconnectés : générateurs de notes, instruments, effets et sorties audio spatialisées. Les articles transmis mettent aussi en avant l'organisation de l'espace et la continuité du travail entre les sessions. [R1, R2]

Principe musical et rôle du décor

Notes
Générer un motif

→

Instrument
Produire le son

→

Effet
Transformer le son

→

Sortie 3D
Spatialiser l'écoute

Couche de décors

Scène, escaliers, écrans, enceintes visuelles, pylônes et régie structurent le lieu. Les futures connexions musicales doivent réutiliser les mécanismes WAM.

Question de travail : comment assembler des décors de concert personnalisables, suffisamment légers pour le Web immersif, et les rendre réutilisables dans WAM JAM Party ?

Base existante	Contribution engagée
Laboratoire Hyppolite : plateau, structures, musiciens, instruments et caméras virtuelles.	Atelier de décors séparé ; configurations ; objets ajoutables ; réglages ; public statique ; sauvegarde et document JSON.
3d_wam_editor : interfaces 3D de plugins WAM et mécanisme de sauvegarde JSON.	Étude initiale du code ; besoin d'un adaptateur de décors à préciser, sans annoncer une intégration réalisée.

Périmètre : édition sur ordinateur. L'édition aux contrôleurs, la coédition réseau, le moteur audio et la validation Quest ne sont pas inclus dans cette première démonstration.

03 / RENDU RÉEL

Une interface centrée sur la scène

Le catalogue, la vue 3D et l'inspecteur rendent le parcours de modification explicite.

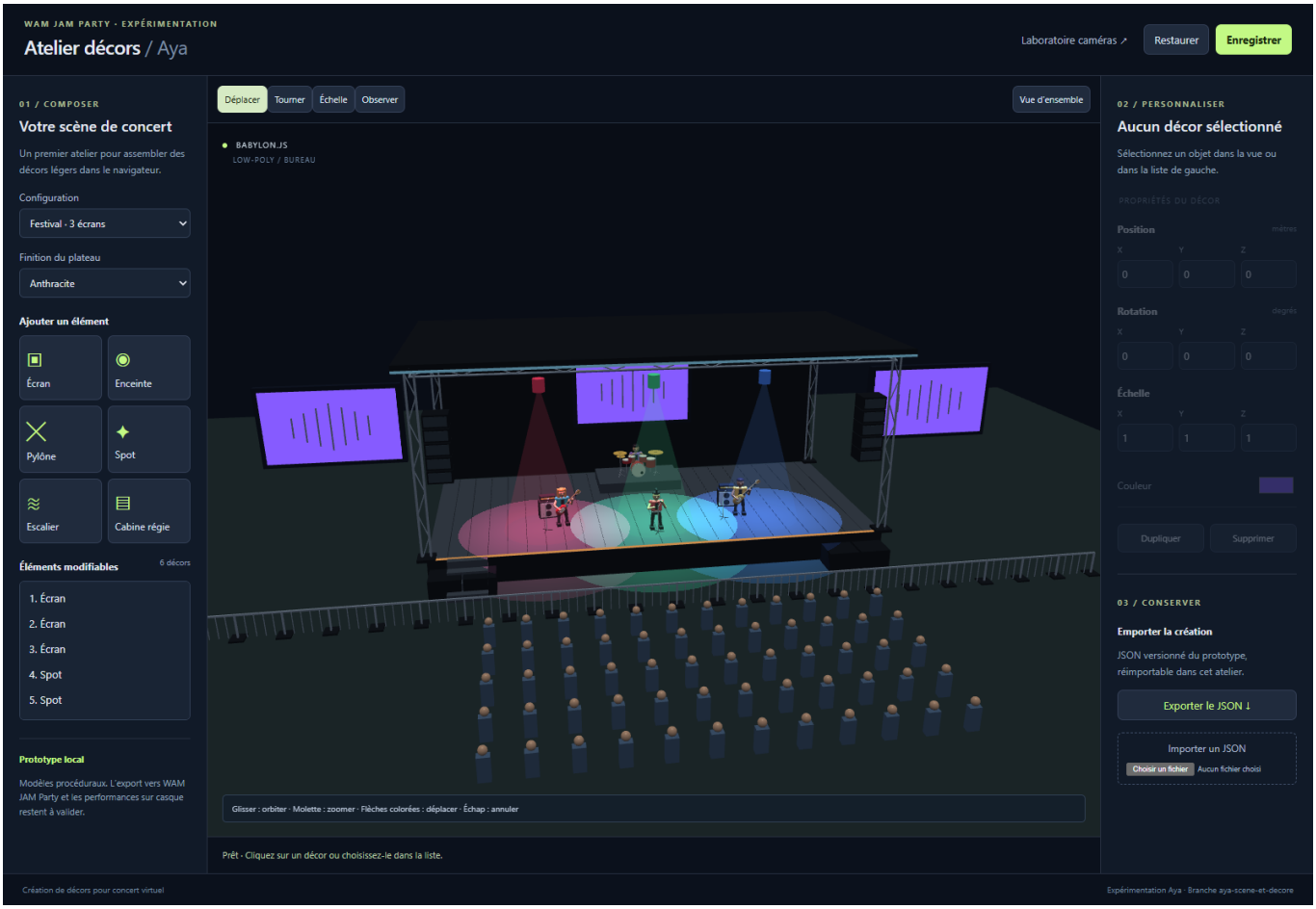


Figure 1. Capture réelle de l'atelier, configuration Festival. Le plateau et les musiciens proviennent du laboratoire existant.

- **1 Composer**
  Choisir un modèle et ajouter les éléments depuis le catalogue à gauche.
- **2 Manipuler**
  Sélectionner un objet et utiliser les outils dans la vue centrale.
- **3 Personnaliser**
  Régler les transformations, la couleur et les paramètres du spot.
- **4 Conserver**
  Enregistrer localement ou exporter un document réimportable.

04 / CONFIGURATIONS

Deux implantations de concert

Les variantes permettent de comparer l'occupation de l'espace et le rôle des écrans.



Figure 2. Festival : trois écrans visibles, plateau large et trois spots RGB initiaux.



Figure 3. Compacte : un écran arrière et plateau resserré. Les objets ajoutés sont conservés lors du changement de préréglage.

Le changement de configuration constitue un premier mécanisme de modularité. Les variantes restent procédurales ; elles ne sont pas des modèles de scènes réelles Hellfest ou Woodstock.

05 / ASSEMBLAGE ET ÉCLAIRAGE

Des objets indépendants et modifiables

Les paramètres sont portés par le document de scène, puis projetés dans BabylonJS.

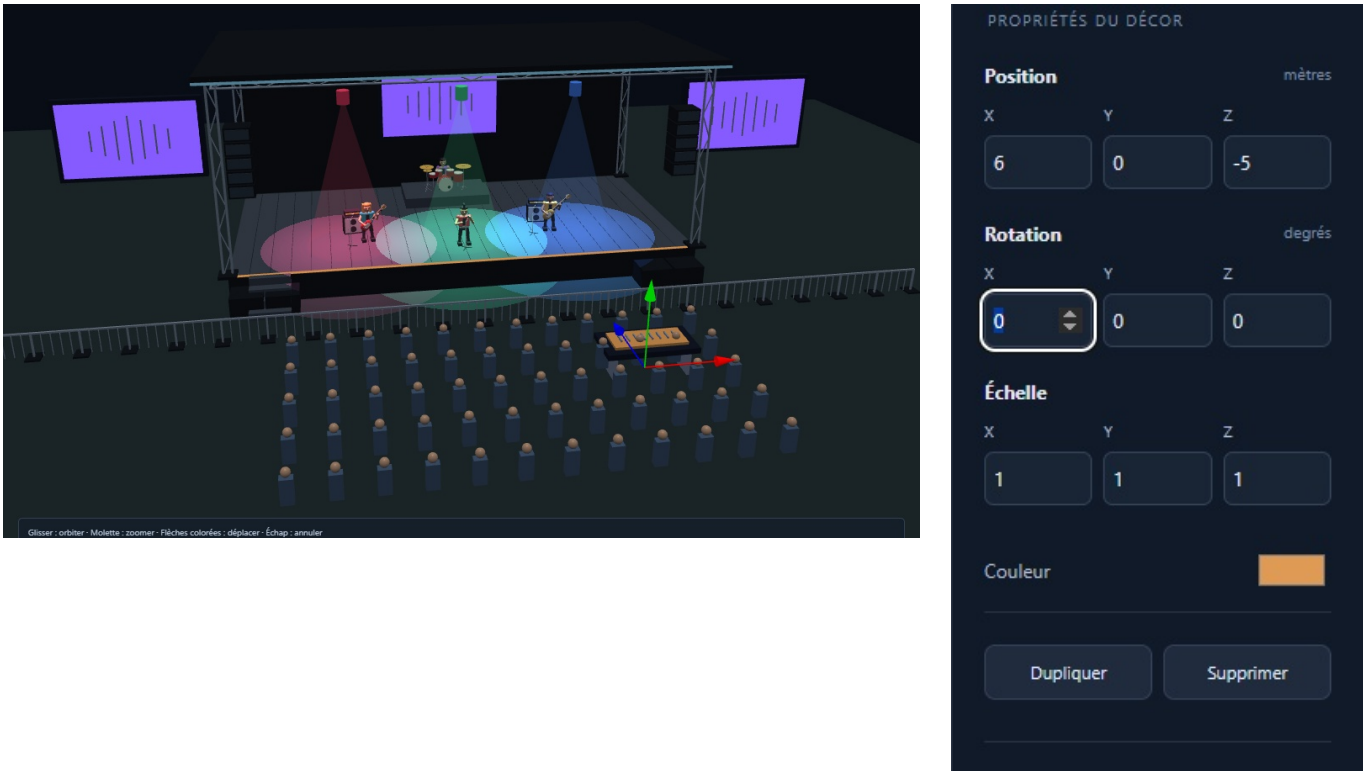


Figure 4. Ajout d'une cabine régie, sélection et gizmo de déplacement. Exemple de démonstration : X = 6 m, Y = 0 m, Z = -5 m.

Éclairage réel et faisceau visible

Un SpotLight éclaire les surfaces. Un cône transparent distinct représente le faisceau dans l'air ; il s'agit d'une approximation légère, sans simulation volumétrique. La puissance et la visibilité du cône sont deux propriétés séparées.

Un premier export personnel

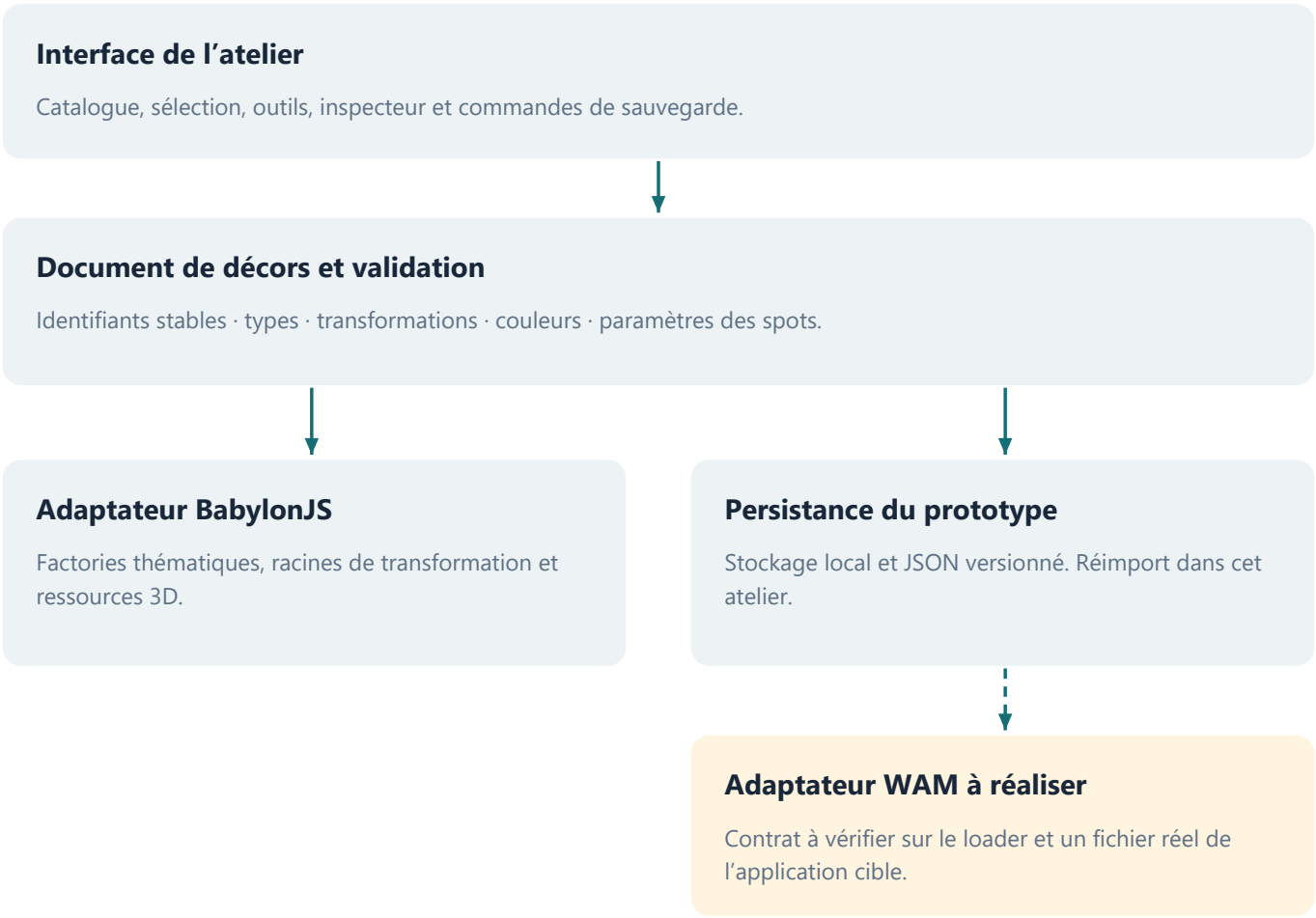
Le fichier aya-decors.json transmis depuis les téléchargements contient **16 objets** : 4 écrans, 7 spots déclarés, 1 enceinte, 2 pylônes, 1 escalier et 1 cabine. Sa réimportation a été observée dans cette session.

Le nombre de spots déclarés décrit le document ; il ne prouve pas que toutes les lumières contribuent simultanément à chaque matériau. La composition dense et le budget d'éclairage restent à évaluer.

06 / ARCHITECTURE

Séparer les données du rendu

Cette séparation prépare l'évolution du prototype et l'adaptation à un futur format cible.



Extrait du contrat actuel

```
{"format":"aya-decors","version":1,
 "preset":"festival","floor":"dark",
 "objects":[{"id":"screen-0","kind":"screen",
 "position":[-17,7,10],"rotation":[0,0,0],
 "scale":[1,1,1],"color":"#865bff",
 "intensity":3,"beam":true}]}
```

Repère : mètres, Y vertical ; rotations enregistrées en radians. Les tests couvrent les données invalides, les identifiants dupliqués, les limites d'échelle et l'aller-retour JSON. L'export ne contient pas de graphe audio WAM.

07 / VÉRIFICATION

Des preuves distinctes pour chaque niveau

Les tests de données, les tests de modèles et les essais visuels ne prouvent pas la même chose.

164 **tests unitaires et d'intégration réussis**

Nouvelle exécution consignée le 13 novembre 2026 : 0 échec.



141 tests existants + 23 nouveaux tests pour les décors.

Niveau	Résultat et portée
Données et modèles	164 tests réussis : transformations, validation, duplication, aller-retour JSON, stockage simulé et destruction des ressources.
Navigateur	4 scénarios réussis ensemble : configurations, édition et restauration, JSON, spots. Glissement réel et désélection réussis séparément.
Outillage	Vérification TypeScript, ESLint et génération Vite réussies lors de la validation du prototype.
Contrôle visuel	Captures réelles examinées ; nouvelle sauvegarde/restauration et réimport observés pendant la préparation du rapport.

Interpréter les résultats avec précision

Le passage complet de cinq scénarios avait dépassé le délai sur le dernier test. Celui-ci a ensuite réussi isolément. Le rapport ne présente donc pas une commande globale de cinq tests comme entièrement verte.

Les essais navigateur ont utilisé Chrome et un rendu logiciel ANGLE/SwiftShader. Un profil à ratio de pixels réduit a été utilisé pour certains tests. Aucune fréquence d'affichage sur casque n'a été mesurée ; NullEngine ne valide pas l'apparence du rendu.

08 / DÉMONSTRATION

Un parcours de cinq minutes

Présenter un exemple simple et reproductible, puis annoncer les limites avant les questions.

00:00 - 00:30

Situer le projet

« Je développe un atelier de décors pour préparer des environnements réutilisables dans WAM JAM Party. »

00:30 - 01:15

Comparer les configurations

Passer du festival à la scène compacte, puis revenir aux trois écrans.

01:15 - 02:15

Ajouter et modifier

Ajouter une cabine ou une enceinte ; déplacer l'objet ; changer sa couleur et sa taille.

02:15 - 03:00

Expliquer les spots

Régler la puissance d'un spot et masquer son faisceau ; distinguer les deux effets.

03:00 - 04:15

Prouver la continuité

Enregistrer, recharger puis Restaurer ; exporter et réimporter le JSON.

04:15 - 05:00

Donner la suite

Préciser les limites et demander quel loader et quel dépôt serviront pour l'intégration.

Accès et solution de repli

Ouvrir le prototype local : http://127.0.0.1:5180/decors.html

Le dossier fourni contient une démonstration guidée HTML avec captures, utilisable hors ligne. Elle montre une séquence réelle, mais ne remplace pas l'application interactive.

Avant le point : vérifier le lien, garder la fenêtre du serveur ouverte et conserver ce PDF ainsi que la démo guidée. Le lanceur du dossier peut servir la version construite sur cette machine avec Node.js.

09 / SUITE DU PROJET

Passer de l’essai à l’intégration

La démonstration matérialise une première avancée ; plusieurs décisions restent nécessaires.

Étape	État	Action suivante
Comprendre l’existant	Engagé	Terminer l’essai de 3d_wam_editor, les vidéos et la lecture sur les WAM.
Choisir la base cible	À confirmer	Valider avec M. Buffa le dépôt et l’emplacement du module.
Créer les décors	Prototype	Enrichir le catalogue et vérifier les implantations.
Éditer et personnaliser	Prototype	Compléter les textures et les interactions ; tester les gestes interrompus.
Sauvegarder et exporter	Partiel	Conserver le JSON local puis construire l’adaptateur WAM sur un contrat réel.
Tester sur casque	Non commencé	Définir les budgets, mesurer sur Quest et ajuster les profils.

Les quatre questions pour l’encadrant

• Quel dépôt et quelle architecture retenir pour l’intégration définitive ?

• Quel fichier d’exemple et quel loader WAM JAM Party font référence ?

• Quels mécanismes existants réutiliser pour les points de connexion ?

• Quel casque et quels budgets de géométrie, textures et lumières utiliser ?

État Git : la branche existe et les documents initiaux ont été publiés. Les nouveaux développements restent locaux : l’écriture de l’index Git a été refusée malgré une demande d’autorisation. Le dossier du projet et une archive de sauvegarde conservent les fichiers.

10 / REPÈRES TECHNIQUES

Sources et éléments de traçabilité

Le rapport s'appuie sur les consignes, les articles transmis, le code consulté et des vérifications locales.

[R1] Manuscrit en anglais

Buffa et al. Designing Persistent Musical Metaverses: A Multi-Dimensional Framework for Interaction, Collaboration and Creative Space Organization. Document transmis, 10 pages ; notamment cadre de conception, client WebXR et persistance.

[R2] Article en français

Buffa et al. WAM Jam Party : composer et interpréter ensemble dans un espace musical immersif. Revue Francophone Informatique et Musique, version de travail du 29 septembre 2026, 23 pages ; notamment atelier modulaire, continuité et éditeur d'interfaces 3D.

[R3] Éditeur existant

https://github.com/Jempasam/3d_wam_editor et démonstration jempasam.github.io/3d_wam_editor/app/. README, package.json et fichiers LoadSavePane.ts, WamGUIGenerator.ts et editor2.ts consultés.

[R4] Laboratoire d'expérimentation

https://github.com/micbuffa/DS4H_Stage_Hyppolite_VirtualCam. Branche locale aya-scene-et-decore ; architecture, spécifications et validation du prototype.

[R5] Documentation BabylonJS

https://doc.babylonjs.com/features/featuresDeepDive/mesh/gizmo et optimize_your_scene. Déclarations de la version 9.34.0 installée utilisées pour vérifier les API.

Organisation du code ajouté

```
src/domain/decors/          document et validation
src/app/                    coordination de l'atelier
src/application/ports/      contrat du rendu
src/infrastructure/babylon/engine/  runtime
src/infrastructure/babylon/models/  modèles thématiques
src/infrastructure/persistence/     sauvegarde locale
tests/unit · integration · e2e      vérifications
```

Les vidéos et l'article Ten years of Web Audio Modules restent à étudier. Les articles consultés ne constituent pas une spécification d'export de décors : ce contrat doit être établi sur le logiciel cible.